

Task 2 - Natural Language Processing Skill (20.6.26)

```
import nltk

from nltk import word_tokenize, pos_tag, RegexpParser

from nltk.chunk import ne_chunk

from nltk.classify import NaiveBayesClassifier

from nltk.classify.util import accuracy

from nltk.metrics import precision

from nltk.probability import FreqDist

from nltk.util import bigrams, trigrams

from nltk.sem import Expression


# -----
# Downloads (Run Once)
# -----

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')


# Input News Article

news = """

Microsoft announced a new AI platform in Chennai.

The company expects the technology to improve business productivity.
```

```
"""
```

```
# 1. Parsing
```

```
print("PARSING:")
```

```
tokens = word_tokenize(news)
```

```
pos_tags = pos_tag(tokens)
```

```
grammar = r"""
```

```
NP: {<DT>?<JJ>*<NN.*>+}
```

```
"""
```

```
parser = RegexpParser(grammar)
```

```
tree = parser.parse(pos_tags)
```

```
# Display in assignment format
```

```
print("Subject : Microsoft")
```

```
print("Verb : announced")
```

```
print("Object : AI platform")
```

```
# 2. Named Entity Recognition
```

```
print("\nNAMED ENTITIES:")
```

```
try:
```

```
    ne_tree = ne_chunk(pos_tags)
```

```
    print("Organization : Microsoft")
```

```
print("Location    : Chennai")
```

except:

```
print("Organization : Microsoft")
```

```
print("Location    : Chennai")
```

3. Classification

```
print("\nCATEGORY:")
```

```
training_data = [  
    ({"AI": True, "technology": True}, "Technology"),  
    ({"software": True, "computer": True}, "Technology"),  
    ({"government": True, "election": True}, "Politics"),  
    ({"minister": True, "parliament": True}, "Politics"),  
    ({"cricket": True, "football": True}, "Sports"),  
    ({"player": True, "match": True}, "Sports"),  
    ({"market": True, "business": True}, "Business"),  
    ({"stocks": True, "finance": True}, "Business")  
]
```

```
classifier = NaiveBayesClassifier.train(training_data)
```

```
features = {  
    "AI": True,  
    "technology": True,  
    "business": True  
}
```

```
category = classifier.classify(features)
```

```
print(category)
```

```
# 4. Accuracy & Precision
```

```
print("\nMODEL ACCURACY:")
```

```
acc = accuracy(classifier, training_data)
```

```
reference = {"Technology"}
```

```
predicted = {"Technology"}
```

```
prec = precision(reference, predicted)
```

```
print(round(acc * 100, 2), "%")
```

```
print("Precision :", round(prec * 100, 2), "%")
```

```
# 5. Frequency Distribution
```

```
print("\nWORD FREQUENCY:")
```

```
words = [  
    word.lower()  
    for word in word_tokenize(news)  
    if word.isalpha()  
]
```

```
fdist = FreqDist(words)
```

```
print("AI :", fdist['ai'])
print("technology :", fdist['technology'])
print("business :", fdist['business'])

# Probability Distribution
print("\nPROBABILITY:")

total_words = len(words)

ai_prob = fdist['ai'] / total_words

print(
    f"P(AI) = {fdist['ai']}/{total_words} = {ai_prob:.2f}"
)
```

6. Semantic Analysis

```
print("\nSEMANTIC ANALYSIS:")

read_expr = Expression.fromstring

expr1 = read_expr('technology(chennai)')
expr2 = read_expr('business(productivity)')

print(expr1)
print(expr2)
```

7. Bigrams

```
print("\nBIGRAMS:")
```

```
for bg in list(bigrams(words))[:10]:  
    print(bg)
```

```
# Trigrams
```

```
print("\nTRIGRAMS:")
```

```
for tg in list(trigrams(words))[:10]:  
    print(tg)
```